

# Cloud Native Qumulo IAM Permissions Tester - Usage Guide

## Overview

The IAM Permissions Tester validates that your AWS credentials have the necessary permissions to deploy Cloud Native Qumulo infrastructure using Terraform. It performs **non-destructive** testing using AWS IAM policy simulation, ensuring no actual resources are created during the validation process.

## Prerequisites

### Installation

```
Shell
pip install boto3 colorama tabulate
```

### AWS Credentials

Ensure you have AWS credentials configured using one of these methods:

- **AWS CLI:** `aws configure`
- **Environment Variables:** `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`
- **IAM Role:** If running on EC2 instance
- **AWS Profiles:** Multiple profile support available

## Basic Usage

### Test All Permissions (Recommended)

```
Shell
python iam_permissions_tester.py
```

This runs a comprehensive test of all permissions required for Cloud Native Qumulo deployment.

## Test with Specific AWS Profile

Shell

```
python iam_permissions_tester.py --profile my-terraform-profile
```

Use this when you have multiple AWS profiles configured and want to test a specific one.

## Test in Different Region

Shell

```
python iam_permissions_tester.py --region us-east-1
```

Specify the AWS region where you plan to deploy the infrastructure.

## Targeted Testing Options

### Core Services Only

Shell

```
python iam_permissions_tester.py --test-type core
```

Tests essential permissions for:

- **EC2**: Instance management, security groups, volumes
- **S3**: Bucket management for persistent storage
- **IAM**: Role and policy management

### Additional Services

Shell

```
python iam_permissions_tester.py --test-type additional
```

Tests supporting service permissions for:

- **Systems Manager**: Parameter store access
- **Secrets Manager**: Secret management
- **CloudWatch**: Logging and monitoring

## Conditional Features

Shell

```
python iam_permissions_tester.py --test-type conditional
```

Tests optional feature permissions for:

- **Route 53 Resolver:** DNS forwarding (if using `q_cluster_fqdn`)
- **Elastic Load Balancing:** Network Load Balancer (if `q_nlb_provision = true`)
- **KMS:** Customer-managed keys (if `kms_key_id` specified)

## Terraform State Management

Shell

```
python iam_permissions_tester.py --test-type state
```

Tests permissions for:

- **S3 Backend:** Terraform state storage
- **DynamoDB:** State locking (if configured)

## Real-World Usage Scenarios

### Pre-Deployment Validation

Shell

```
# Before running terraform plan/apply
python iam_permissions_tester.py --profile prod-terraform
--region us-west-2
```

### Troubleshooting Failed Deployments

Shell

```
# If Terraform deployment fails with permission errors
python iam_permissions_tester.py --test-type core
```

### Validating New IAM Policies

Shell

```
# After updating IAM policies, validate they're sufficient
python iam_permissions_tester.py --profile updated-role
```

## CI/CD Pipeline Integration

Shell

```
# In CI/CD pipeline before Terraform execution
python iam_permissions_tester.py || exit 1
```

## Understanding the Output

### Success Indicators

- **✓ Green checkmarks:** Permissions are available
- **High success rates:** 90%+ indicates good permission coverage
- **"All permission tests passed!":** Ready for deployment

### Warning Signs

- **✗ Red X marks:** Missing critical permissions
- **⚠ Yellow warnings:** Simulation errors or throttling
- **Low success rates:** <80% indicates significant permission gaps

### Generated IAM Policy

The tool automatically generates an IAM policy for any missing permissions:

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:CreateSecurityGroup",
        "s3:CreateBucket"
      ]
    }
  ]
}
```

```
    ],  
    "Resource": "*"    
  }  
]  
}
```

## Common Use Cases

### 1. New Environment Setup

When setting up Cloud Native Qumulo in a new AWS account:

Shell

```
python iam_permissions_tester.py --profile new-account-admin
```

### 2. Least Privilege Validation

After creating minimal IAM roles for Terraform:

Shell

```
python iam_permissions_tester.py --profile terraform-minimal-role
```

### 3. Deployment Type Validation

For deployments with specific features:

Shell

```
# Standard deployment (no optional features)  
python iam_permissions_tester.py --test-type core --test-type  
additional  
  
# Deployment with Network Load Balancer
```

```
python iam_permissions_tester.py # Tests all including
conditional

# Deployment with custom KMS keys
python iam_permissions_tester.py --test-type conditional
```

## 4. Multi-Region Validation

Testing the same role across different regions:

```
Shell
python iam_permissions_tester.py --region us-west-2
python iam_permissions_tester.py --region us-east-1
```

# Best Practices

## 1. Run Before Every Deployment

```
Shell
# Add to deployment scripts
python iam_permissions_tester.py --profile $AWS_PROFILE --region
$AWS_REGION
if [ $? -eq 0 ]; then
    echo "Permissions validated. Proceeding with deployment..."
    terraform apply
else
    echo "Permission validation failed. Please review IAM
policies."
    exit 1
fi
```

## 2. Document Required Permissions

Save the generated IAM policy output for your organization's IAM policy documentation.

### 3. Regular Validation

Run periodic checks to ensure IAM policies haven't been inadvertently modified:

Shell

```
# Weekly validation in production environments
python iam_permissions_tester.py --profile prod-terraform
```

### 4. Environment-Specific Testing

Use different test types based on your deployment configuration:

- **Development:** `--test-type core` (minimal features)
- **Staging:** `--test-type all` (full feature set)
- **Production:** `--test-type all` (comprehensive validation)

## Troubleshooting

### Rate Limiting Issues

If you encounter throttling errors:

Shell

```
# Test in smaller chunks
python iam_permissions_tester.py --test-type core
sleep 30
python iam_permissions_tester.py --test-type additional
```

### Simulation Errors

Some IAM principals (like certain service roles) can't be simulated:

- Review the detailed error output
- Manually verify permissions if simulation fails
- Consider using a different IAM principal for testing

### Missing Permissions

Use the generated IAM policy to update your Terraform execution role:

1. Copy the generated policy JSON
2. Attach it to your IAM role/user
3. Re-run the test to verify

## Exit Codes

- **0**: All tests passed - ready for deployment
- **1**: Some tests failed - review permissions before deploying

This makes the tool suitable for automated deployment pipelines and CI/CD integration.

## Support

For issues related to the tool itself, ensure you have:

- Valid AWS credentials configured
- Sufficient IAM permissions to call `iam:SimulatePrincipalPolicy`
- Network connectivity to AWS APIs
- Python dependencies installed correctly